

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA
Grado en Ingeniería Informática

Interfaces y Periféricos

Proyecto Final: Sistema de Desactivación de Explosivos

Autores:

Antonio CAÑETE LÓPEZ

Manuel REYES SERRANO

Asignatura:

Interfaces y Periféricos

Plataforma:

Tinkercad

22 de mayo de 2026

Índice

1. Introducción	2
1.1. El Juego	2
2. Materiales y Arquitectura	2
3. Modos de Funcionamiento	3
4. Imágenes del Circuito	3
5. Preguntas Frecuentes y Decisiones de Diseño	6
5.1. ¿Por qué se emplea la configuración LiquidCrystal_I2C lcd(0x20, 16, 2)?	6
5.2. ¿Por qué los pulsadores no disponen de resistencias físicas en la protoboard?	7
5.3. ¿Cómo se gestiona el temporizador de forma no bloqueante mediante la variable <code>contadorMuestrasCiclo</code> ?	7
6. Código Fuente	7
6.1. Código Placa 1 (Control Central)	7
6.2. Código Placa 2 (Generador de Secuencia)	11
7. Bibliografía	12

1. Introducción

Proyecto final de la asignatura Interfaces y Periféricos realizado en la web tinkercad en la cual se implementa un juego de desactivación de explosivos.

1.1. El Juego

El juego consta de una "bomba", la cual debemos desactivar según una combinación de botones aleatoria generada por el sistema. Si se acierta la combinación en el orden correcto, saldrá un mensaje por pantalla (LCD) indicando la victoria. En caso de fallar o quedarse sin tiempo, la simulación entrará en estado de detonación y se activará un buzzer continuo.

2. Materiales y Arquitectura

Para el montaje de este proyecto se han utilizado dos microcontroladores trabajando en conjunto y los siguientes periféricos:

- 2 Arduinos UNO.
- 3 Protoboards.
- 5 Botones (configurados en pull-up interno).
- 8 Leds (Azul, Amarillo, Rojo, Verde para estado; 4 Rojos para contraseña).
- Resistencias de 330 ohms.
- 1 Potenciómetro.
- 1 Buzzer (Piezoeléctrico).
- 1 Pantalla LCD I2C (LiquidCrystal_I2C 0x20, 16x2).

La arquitectura se divide en dos placas: La **Placa 1 (Superior)** controla la lógica de la bomba, el LCD, el potenciómetro, el buzzer y recibe los *inputs* de los botones de desactivación (pines digitales 8, 9, 10, 11). Los LEDs conectados a sus pines 2, 3, 4 y 5 muestran el estado actual del juego. La **Placa 2 (Inferior)** se encarga de generar una secuencia aleatoria y mostrarla parpadeando en sus 4 LEDs (pines 3, 4, 5, 6). Esta placa está conectada a la Placa 1 a través de comunicación Serial (del TX de la placa 2 al RX de la placa 1) para enviarle la contraseña correcta.

3. Modos de Funcionamiento

El sistema transiciona por los siguientes estados:

- **Modo Armado:** Estado inicial. El LED azul se enciende. Mediante el potenciómetro se utiliza la función `map` para ajustar el tiempo de cuenta atrás antes de la detonación (de 10 a 60 segundos). Al pulsar el botón de inicio, se envía el byte de control 'y' a la segunda placa para que genere la contraseña.
- **Modo Desactivando:** La Placa 2 genera 5 números aleatorios, los muestra visualmente y los transmite. La Placa 1 apaga el LED azul y enciende el amarillo. El LCD muestra el tiempo restante, restando un segundo cada ciclo mientras el buzzer emite un pitido. El jugador debe introducir la clave correcta.
- **Modo Explosión (KABOOM):** Si el tiempo llega a cero o el jugador se equivoca de botón, se enciende el LED rojo. El LCD muestra "KABOOOOOOOM!!! PERDISTE!!" y el buzzer emite un sonido grave constante.
- **Modo Victoria:** Si la contraseña es introducida correctamente a tiempo, se enciende el LED verde. El LCD muestra "BOMBA DES-ACTIVADA VICTORIA!!" y suena una melodía de éxito.

4. Imágenes del Circuito

A continuación se muestran los esquemas visuales de la simulación en Tinkercad en sus distintas fases de ejecución.

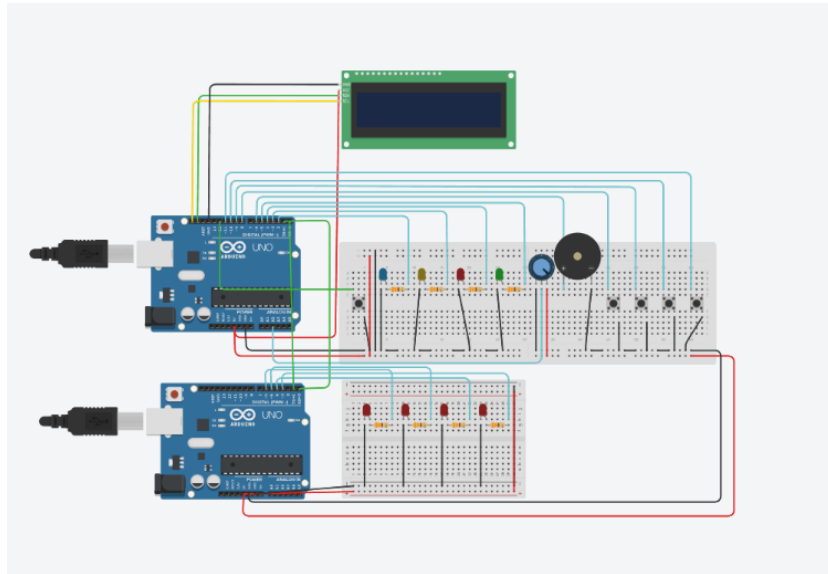


Figura 1: Circuito en estado de reposo (Apagado).

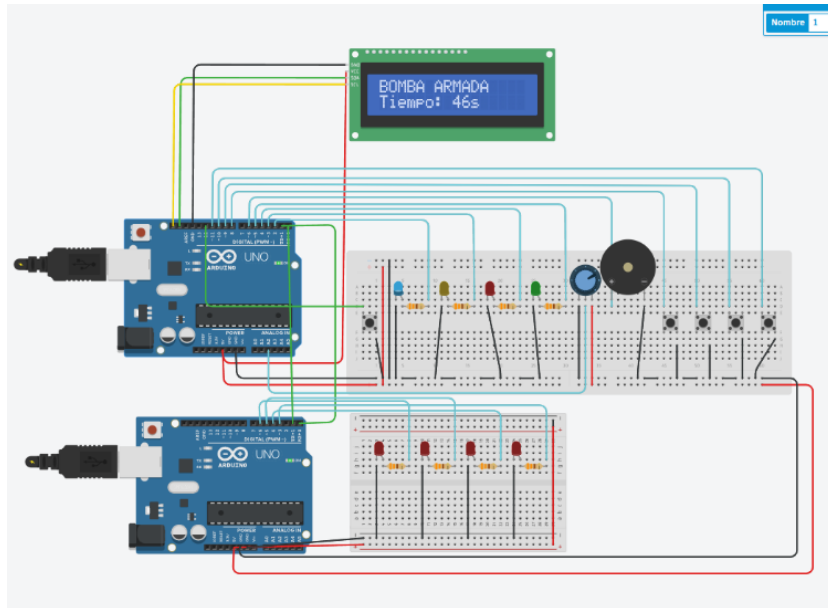


Figura 2: Circuito en Modo ARMADO (LED Azul encendido, LCD activo).

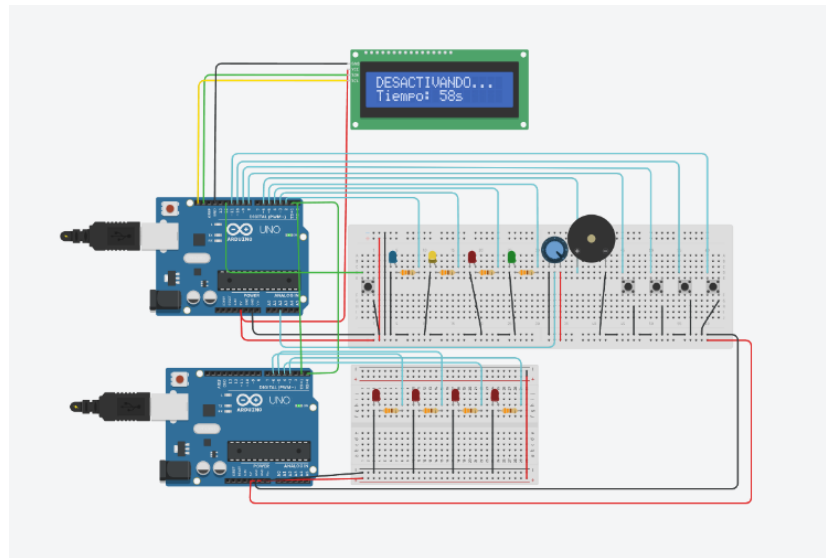


Figura 3: Circuito en Modo DESACTIVANDO (LED Amarillo, temporizador activo).

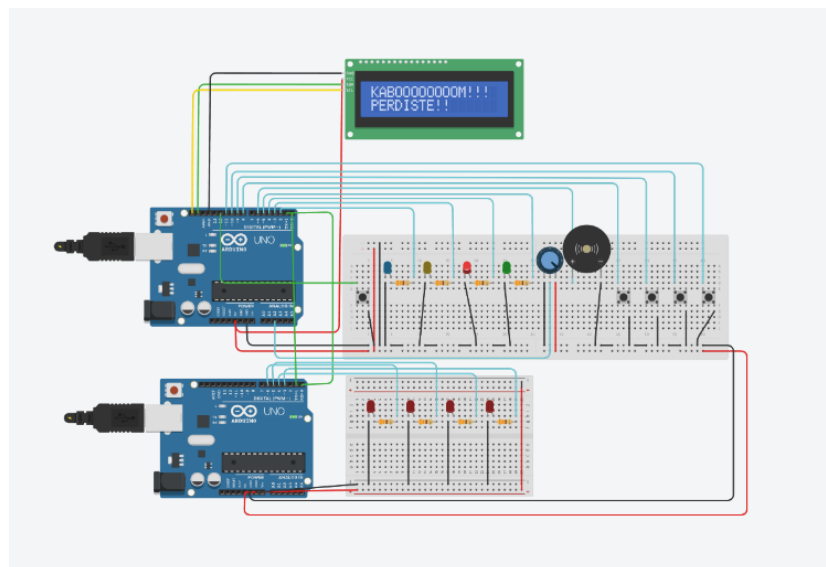


Figura 4: Circuito en Modo EXPLOSIÓN (LED Rojo, fin del juego).

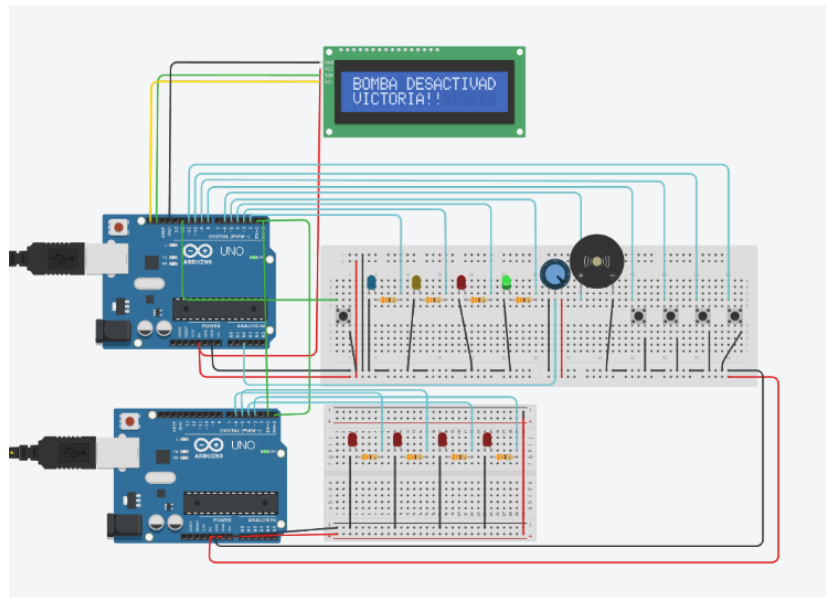


Figura 5: Circuito en Modo VICTORIA (LED Verde, bomba desactivada).

5. Preguntas Frecuentes y Decisiones de Diseño

A continuación, se justifican técnicamente algunas de las soluciones implementadas en el circuito para optimizar recursos y código:

5.1. ¿Por qué se emplea la configuración LiquidCrystal_I2C lcd(0x20, 16, 2)?

La utilización de una pantalla LCD con protocolo de comunicación I2C permite controlar el periférico utilizando únicamente dos pines analógicos dedicados al bus de datos y reloj (SDA y SCL), además de las líneas de alimentación. Esto evita la necesidad de incluir más placas de expansión o saturar las salidas digitales del microcontrolador principal, logrando un diseño limpio y optimizado.

5.2. ¿Por qué los pulsadores no disponen de resistencias físicas en la protoboard?

Para maximizar el espacio útil en las protoboards y reducir la complejidad del cableado, se ha prescindido del uso de resistencias de acoplamiento externas. En su lugar, los pines digitales de entrada conectados a los botones se han configurado mediante software utilizando el modo `INPUT_PULLUP`. De este modo se activan las resistencias de *pull-up* internas de la placa Arduino, manteniendo un estado lógico alto (`HIGH`) por defecto y registrando un estado bajo (`LOW`) únicamente al ser presionados.

5.3. ¿Cómo se gestiona el temporizador de forma no bloqueante mediante la variable `contadorMuestrasCiclo`?

Con el fin de mantener una lectura fluida y constante de los pulsadores sin congelar la ejecución del hilo principal con funciones bloqueantes, el programa aplica un retardo mínimo de 100 ms al final de cada ciclo del `loop()`. La variable `contadorMuestrasCiclo` actúa como un acumulador interno que incrementa en cada iteración. Al alcanzar el valor de 10, el sistema determina de forma precisa que ha transcurrido un segundo real completo, procediendo a restar una unidad al tiempo restante de la bomba y activando el pulso sonoro del buzzer.

6. Código Fuente

6.1. Código Placa 1 (Control Central)

El siguiente código se ejecuta en el Arduino UNO encargado de la lógica principal, lectura de sensores, y control del LCD.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

const int LED_AZUL = 2;
const int LED_AMARILLO = 3;
const int LED_ROJO = 4;
const int LED_VERDE = 5;
```



```

const int PIN_BUZZER = 6;
const int PIN_POTENCIOMETRO = A2;
const int BOTON_START = 12;
const int BOTONES[] = {8, 9, 10, 11};
const int PASOS = 5;

LiquidCrystal_I2C lcd(0x20, 16, 2);

int secuenciaObjetivo[PASOS];
int pasoActual = 0;
int tiempoBomba = 0;
int tiempoRestante = 0;
int contadorMuestrasCiclo = 0;
int estadoJuego = 0;

void setup() {
  Serial.begin(9600);
  lcd.init();
  lcd.backlight();
  pinMode(LED_AZUL, OUTPUT);
  pinMode(LED_AMARILLO, OUTPUT);
  pinMode(LED_ROJO, OUTPUT);
  pinMode(LED_VERDE, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(BOTON_START, INPUT_PULLUP);
  for (int i = 0; i < 4; i++) {
    pinMode(BOTONES[i], INPUT_PULLUP);
  }
}

void loop() {
  // BOMBA ARMADA
  if (estadoJuego == 0) {
    digitalWrite(LED_AZUL, HIGH);
    digitalWrite(LED_AMARILLO, LOW);
    digitalWrite(LED_ROJO, LOW);
    digitalWrite(LED_VERDE, LOW);
  }
}

```

```

int lecturaPot = analogRead(PIN_POTENCIOMETRO);
tiempoBomba = map(lecturaPot, 0, 1023, 10, 60);
lcd.setCursor(0, 0);
lcd.print("BOMBA ARMADA    ");
lcd.setCursor(0, 1);
lcd.print("Tiempo: ");
lcd.print(tiempoBomba);
lcd.print("s    ");

if (digitalRead(BOTON_START) == LOW) {
  Serial.print('y');
  int leidos = 0;
  while (leidos < PASOS) {
    if (Serial.available() > 0) {
      secuenciaObjetivo[leidos] = Serial.read();
      leidos++;
    }
  }
  tiempoRestante = tiempoBomba;
  pasoActual = 0;
  contadorMuestrasCiclo = 0;
  lcd.clear();
  estadoJuego = 1;
}
}
// DESARMANDO
else if (estadoJuego == 1) {
  digitalWrite(LED_AZUL, LOW);
  digitalWrite(LED_AMARILLO, HIGH);
  lcd.setCursor(0, 0);
  lcd.print("DESACTIVANDO... ");
  lcd.setCursor(0, 1);
  lcd.print("Tiempo: ");
  lcd.print(tiempoRestante);
  lcd.print("s    ");

  contadorMuestrasCiclo++;
  if (tiempoRestante <= 0) {

```

```

    estadoJuego = 2;
}

if (contadorMuestrasCiclo >= 10) {
    tiempoRestante--;
    tone(PIN_BUZZER, 1000, 100);
    contadorMuestrasCiclo = 0;
}

for (int i = 0; i < 4; i++) {
    if (digitalRead(BOTONES[i]) == LOW) {
        if (i == secuenciaObjetivo[pasoActual]) {
            pasoActual++;
            tone(PIN_BUZZER, 1500, 150);
            if (pasoActual >= PASOS) {
                estadoJuego = 3;
            }
        } else {
            estadoJuego = 2;
        }
        delay(200);
    }
}

// EXPLOSION
else if (estadoJuego == 2) {
    digitalWrite(LED_AMARILLO, LOW);
    digitalWrite(LED_ROJO, HIGH);
    lcd.setCursor(0, 0);
    lcd.print("KABOOOOOOOOM!!! ");
    lcd.setCursor(0, 1);
    lcd.print("PERDISTE!! ");
    tone(PIN_BUZZER, 150, 2000);
    while (true) {}
}

// VICTORIA
else if (estadoJuego == 3) {
    digitalWrite(LED_AMARILLO, LOW);

```

```

    digitalWrite(LED_VERDE, HIGH);
    lcd.setCursor(0, 0);
    lcd.print("BOMBA DESACTIVAD");
    lcd.setCursor(0, 1);
    lcd.print("VICTORIA!!");
    tone(PIN_BUZZER, 700, 200);
    delay(200);
    tone(PIN_BUZZER, 1200, 400);
    while (true) {}
  }
  delay(100);
}

```

6.2. Código Placa 2 (Generador de Secuencia)

Código implementado en el segundo Arduino encargado de la aleatoriedad y visualización de la secuencia.

```

const int LEDS[] = {6, 5, 4, 3};
const int PASOS = 5;

void setup() {
  Serial.begin(9600);
  for (int i = 0; i < 4; i++) {
    pinMode(LEDS[i], OUTPUT);
  }
  randomSeed(analogRead(A5));
}

void loop() {
  if (Serial.available() > 0) {
    char comando = Serial.read();
    if (comando == 'y') {
      int secuencia[PASOS];

      for (int i = 0; i < PASOS; i++) {
        secuencia[i] = random(0, 4);
      }
    }
  }
}

```

```

    for (int i = 0; i < PASOS; i++) {
        delay(200);
        digitalWrite(LEDs[secuencia[i]], HIGH);
        delay(600);
        digitalWrite(LEDs[secuencia[i]], LOW);
    }

    for (int i = 0; i < PASOS; i++) {
        Serial.write(secuencia[i]);
    }
}
}
}

```

7. Bibliografía

Referencias

- [1] *Tinkercad*: <https://www.tinkercad.com/>
- [2] *Repositorio del proyecto*: https://github.com/i32caloa/ProyectoFinal_IyP
- [3] *Arduino. Referencia del lenguaje Arduino (Funciones, Variables y Estructura)*. Disponible en: <https://www.arduino.cc/reference/es/>